Module	**SE101.3 -OOP with Java – 25.3**
Lab sheet Number	**09**
Areas covered	**Inheritance + Encapsulation + Polymorphism + Abstraction**

Question 1

a) Create a class Product with private productCode, price, and quantity.

b) Provide getters/setters, where **setPrice**() and **setQuantity**() reject negative values.

c) Add **getTotalValue**() returning price × quantity.

d) In main, create a Product, try an invalid price, and show validation works.

Question 2

a) Create a class called Staff with **name** and **basicSalary**, and **calculateSalary**() returning basicSalary.

b) Create subclass Supervisor adding **teamSize**, overriding **calculateSalary**() to add a bonus of Rs. 1000 per team member. Demonstrate both in main.

Question 3

a) Create an interface PaymentMethod with **processPayment(double amount).**

b) Implement it in CreditCard, PayPal, and CashPayment, each printing a different message.

c) In main, build an array of PaymentMethod references holding all three, and call **processPayment()** in a loop.

Question 4

a) Create an interface called "Shape" with two abstract methods: "**double calculateArea()"** and "**double calculatePerimeter()".**

b) Implement the "Shape" interface in three classes: "**Circle**", "**Rectangle**", and "**Triangle**".

c) Each class should have private instance variables relevant to its shape and provide public getter and setter methods for these variables.

d) Additionally, each class should define a constructor that initializes the instance variables.

e) Write the implementation code for the "Shape" interface, the getter and setter methods in each class, and the constructors in each class.

Question 5

a) Create an abstract class Account with attributes accountNumber and balance, and abstract methods deposit(double amount) and withdraw(double amount).

b) Implement subclasses SavingsAccount and CheckingAccount with their own specific implementations of deposit and withdraw.

c) Demonstrate how each account type handles transactions.

Question 6

Create a program with a mix of inheritance and interfaces to represent different types of electronic devices:

1. ElectronicDevice Interface:
    - Define a method turnOn() that prints a message indicating the device is turning on.
2. Rechargeable Interface:

- Define a method chargeBattery() that prints a message indicating the device is charging.

3. MobileDevice Class:
    - This class implements both ElectronicDevice and Rechargeable.
    - Add attributes brand and batteryCapacity. Implement turnOn() and chargeBattery() methods to display appropriate messages.
4. CameraCapable Interface:
    - Define a method takePhoto() that prints a message indicating the device is taking a photo.
5. Smartphone Class:
    - This class extends MobileDevice and implements CameraCapable.
    - Add attributes operatingSystem and cameraResolution. Implement **takePhoto**() to display an appropriate message.

In the main method, create an instance of Smartphone, set its attributes, and call all methods (**turnOn**(), **chargeBattery**(), and **takePhoto**()).

Question 7

Build a small university system:

a. abstract class Person (private name, id, abstract getRole());

b. interface Payable (calculatePayment());

c. subclass Lecturer extends Person and implements Payable (pay = hoursTaught × rate); subclass Student extends Person only.

d. In main, loop through a Person[] array calling getRole() polymorphically, then call calculatePayment() only on the Lecturer.